BROWN
Computer Science

CS1010: Theory of Computation

Lecture 4: The Pumping Lemma

Lorenzo De Stefani

Fall 2019

Outline

- Non-regular languages
- The Pumping Lemma
  - Main idea
  - Proof of the Lemma
  - Using the Lemma

From Sipser Chapter 1.4

Non-Regular Languages

- Do you think every language is regular?
  - That would mean that every language can be described by a Finite States Automata
- What might make a language non-regular?
  - Hint: Think about the properties of a finite automata.
  - Answer: finite memory
  - Rule of thumb: a language is not regular if you need infinite memory

Some Examples

- Are the following languages regular?
  - L1 = {w| w has an equal number of 0's and 1's}
  - L2 = {w| w has at least 100 1's}
  - L3 = {w| w is of the form 0^n1^n, n≥ 0}
  - L4 = {w|W has the same number of "01" and "10"}
- First, write out some of the elements in each to ensure you have the terminology down
  - L1 = {ε, 01, 10, 1100, 0011, 0101, 1010, 0110, …}
  - L2 = {(100 1's), 0(100 1's), 1(100 1's), …}
  - L3 = {ε, 01, 0011, 000111, …}
  - L4 = {ε,0,00,010,101,11,111,000,…..}

Answers

- All previous languages are infinite!
  - L1 and L3 require infinite memory
  - L2 and L4 only require finite memory
- Memory is the discriminant
  - Languages using finite memory are regular
  - Languages using infinite memory are not regular
  - Finite languages always use finite memory → They are always regular!

A language is not regular if requires infinite memory

Implications of finite memory

Regular languages can be infinite but must be described using finite number of states

- Thus there are restrictions on the structure of regular languages
- For a FA to generate an infinite set of string, clear there must be a _____ between some states

Implications of finite memory

Regular languages can be infinite but must be described using finite number of states

- Thus there are restrictions on the structure of regular languages
- For a FA to generate an infinite set of string, clear there must be a loop between some states

- This leads to the pumping lemma

Pumping Lemma for Regular Languages

- The pumping lemma states that all regular languages have a special property
- If a language does not have this property then it is not regular
  - So can use to prove a language non-regular
  - Be careful of the direction of the implication: the pumping lemma can hold and a language still not be regular. This is not usually highlighted.

Pumping Lemma

If A is a regular language, there is a value *p* – the pumping length- such that any string s in A with length at least p can be divided in three parts s=xyz such that:

1. For each $i \geq 0$, $x\, y^i\, z \in L$
2. $|y| > 0$, and
3. $|xy| \leq p$

Careful about the quantifiers

The property is saying that:

- There exists at least one value for the pumping length for which it holds
  - Just because the PL does not hold for a specific value of the pumping length we cannot conclude that the language is not regular
- The property must hold for all strings of length at least p
  - Just because it holds for some we cannot conclude that it is regular!
- For a given string there must exist at least one possible way of splitting it in xyz (while respecting the constraint) such that xy^iz is in the language itself
  - Just because one possible way does not allow to “pump” y while remaining in the language, we cannot conclude that the language is not regular!

What does this mean?

- Every string in a regular language L with length greater than the pumping length *p* can be "pumped"
- This means every string $s \in$ L contains a section (y) that can be repeated (pumped) any number of times (via a loop)

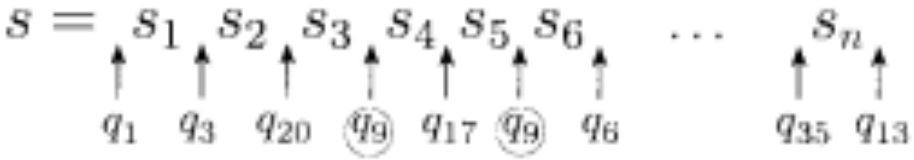
In detail

- Condition 1: for each $i \geq 0$, $x\, y^i\, z \in L$
  - This just says that there is a loop
- Condition 2: $|y| > 0$
  - Without this condition, then there really would be no loop
- Condition 3: $|xy| \leq p$
  - We don't allow more states than the pumping length, since we want to bound the amount of memory
- The conditions allow x or z to be ε
  - So loop need not be in the middle, which would be limiting

Pumping Lemma Proof Idea

Set the pumping length *p* to number of states of the FA

- If length if all strings in A have length lower than p the statement is trivially true
- For |c|≥ p, consider the states that the FA goes through for s

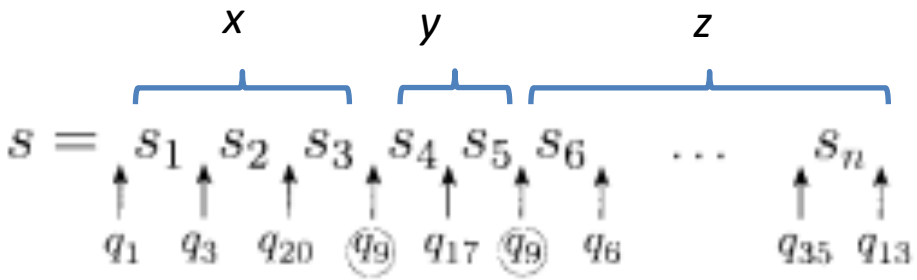


Pidgeonhole principle: Since there are only p states and length s > p, by pigeonhole property one state much be repeated

→ This means there is a cycle!!!

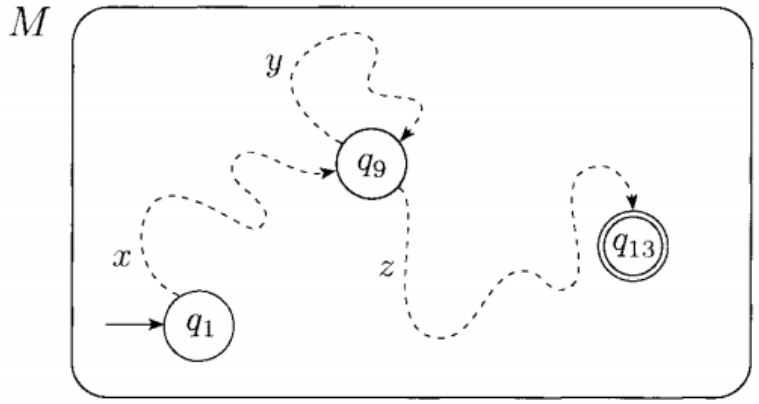
Pumping Lemma Proof continued

Let us divide *s* as:



Pumping lemma properties for *x*, *y* and *z* are verified

Simplified representation of the DFA



*xy*z* will be accepted as well!!!

Meaning of the PL

FA can only use finite memory. If infinite strings, then the loop must handle this

- If there are two parts that can generate infinite sequences, we must find a way to link them in the loop
- If not, the language is not regular

How to use the PL

We use the Pumping Lemma to prove that a given language A is NOT regular

- Proof by contradiction:
  1. Assume A is regular → must satisfy the PL for a certain pumping length *p*
  2. Find one string *s* in A, with $|s| \geq p$, which cannot be "pumped"
     - Consider all possible ways of partitioning *s* in *x,y,z* which satisfy the condition of the PL
     - Show that $xy^iz \notin A$
  3. PL is violated → Contradiction!

Careful about the quantifiers

Given a language A:

- We have to argue that there is no possible value of the pumping length p for which the statement holds
  - Just because the PL does not hold for a specific value of the pumping length we cannot conclude that the language is not regular
- For a given value of the pumping length p showing a single string which is problematic is sufficient!
- For a given "problematic string" *s*, we have to show that there is no possible way of partitioning it in xyz (while following the constraints) such that xy^iz is in the language A as well
  - Just because one possible way does not allow to "pump" y while remaining in the language, we cannot conclude that the language is not regular!

Example 1

Let B be the language $\{0^n1^n \mid n \geq 0\}$ (Ex 1.73)

- Prove B is not regular
- We will use proof by contradiction. Assume B is regular. Now pick a string that will cause a problem.
- What string? Use the pumping length *p* in the string.
- Try 0^p1^p
  - We need $x\,y^i\,z$, let's focus on y.
    - If y all 0's or all 1's, then if $xyz \in L$ then $xyyz \notin L$
    - If y a mixture of 0 and 1, then 0's and 1's in s not completely separated
    - But even simpler than this if use condition 3, which you should
      - » Then y must be all 0's and hence "pumping up" leads to a contradiction

Example 2

Let C = {w | w has equal number of 0's and 1's} (Ex 1.74)

- Can you do this with finite memory?
  - No
- Prove C is not regular, using proof by contradiction
- Assume C is regular. Pick a problematic string.
  - Let's try 0^p1^p
  - If we pick y =01, can we pump it and have pumped string $\in$ C?
    - If |xy| < p then x and y can only be 0s, since we start with 0^p
    - So y can only have 0s and pumping break equality of number of zero and ones

Example 3: Palindromes

Let F = {ww| w $\in$ {0,1}*} (Ex 1.75)

- F = {ε, 00, 11, 0011, 0101, …}
- Can you do this with finite memory?
  - No
- Use proof by contradiction. Pick s $\in$ F that will be problematic
  - 0^p10^p1
    - If x and z could be ε, then easy (use $y=0^p10^p1$)
    - Since |xy| < p, y must be all 0's
    - If we pump y, then only adding 0's. That will be a problem. Since 0's separated by 1 must be equal

Example 4

$D = \{1^{n^2} \mid n \geq 0\}$

- $D = \{\varepsilon, 1, 1111, 11111111, \ldots\}$
- Proof by contradiction
- Choose 1^{p^2}
  - Assume we have an $xyz \in D$
  - What about xyyz? The # of 1's differs from xyz by $|y|$
    - Since $|xy| \leq p$ then $|y| \leq p$
    - Thus xyyz has at most p more 1's than xyz
    - So if xyz has length $\leq p^2$ then xyyz $\leq p^2 + p$
    - But $p^2 + p < (p+1)^2 = p^2 + 2p + 1$
    - Thus length of xyyz lies between consecutive perfect squares and hence $xyyz \notin D$